International Journal of Machine Learning and Cybernetics (2021) 12:231–241
https://doi.org/10.1007/s13042-020-01167-7

ORIGINAL ARTICLE

Multi-agent reinforcement learning for redundant robot control in task-space

Adolfo Perrusquía[1] · Wen Yu[1] · Xiaoou Li[2]

Received: 27 February 2020 / Accepted: 29 June 2020 / Published online: 9 July 2020
© Springer-Verlag GmbH Germany, part of Springer Nature 2020

Abstract
Task-space control needs the inverse kinematics solution or Jacobian matrix for the transformation from task space to joint space. However, they are not always available for redundant robots because there are more joint degrees-of-freedom than Cartesian degrees-of-freedom. Intelligent learning methods, such as neural networks (NN) and reinforcement learning (RL) can learn the inverse kinematics solution. However, NN needs big data and classical RL is not suitable for multi-link robots controlled in task space. In this paper, we propose a fully cooperative multi-agent reinforcement learning (MARL) to solve the kinematic problem of redundant robots. Each joint of the robot is regarded as one agent. The fully cooperative MARL uses a kinematic learning to avoid function approximators and large learning space. The convergence property of the proposed MARL is analyzed. The experimental results show that our MARL is much more better compared with the classic methods such as Jacobian-based methods and neural networks.

Keywords Multi-agent · Reinforcement learning · Redundant robot

1 Introduction

Task-space control[1] is an important job in robotics and robot control, where the robot end-effector is forced to reach the desired reference position and orientation in the workspace [41]. Since the control action is in joint-space, classic control approaches require the inverse kinematics or velocity kinematics (Jacobian matrix). Redundant robots have more joint angles than the robot degree-of-freedom (DOF) of the end-effector [26]. These extra DOFs help the robot to achieve more complex tasks, such as the human arm. The DOF of redundant robots is high, and it is very difficult to obtain an accurate representation of the inverse kinematics solution and the Jacobian matrix [16]. Even more, the classical kinematic methods are developed for non-redundant robots [1, 15], and are not applicable to a redundant one, because there exists multiple solutions [14, 25]. The inverse and velocity kinematics problem is also known as the kinematic problem of redundant robots.

There exists several solutions to the kinematic problem, where the most famous approaches are at a velocity level [24] using the Jacobian matrix. It is well known that for a redundant robot the Jacobian is not a square matrix, so the space transformation from task to joint space cannot be applied directly. The existing methods are based on the pseudo-inverse method which could have controllability problems if the robot is at a singular point [3, 4, 9]. Approximate solutions like singularity avoidance method is used to solve the controllability problem, nevertheless it may have high or very small joint rates that affects the behavior of the controller [26]. Some approaches use augmented models that use components of the end-effector orientation [44] to ensure asymptotic stability of the end-effector, however it requires the dynamic model of the robot which is difficult to obtain.

Learning algorithms [46], such as neural networks are used to solve the inverse kinematic problem in [10]. These methods learn a specific trajectory [5, 33] or an idealized kinematic model [13]. Reinforcement learning (RL) has

✉ Wen Yu
yuw@ctrl.cinvestav.mx

1 Departamento de Control Automático, CINVESTAV-IPN (National Polytechnic Institute), Mexico City, Mexico

2 Departamento de Computación, CINVESTAV-IPN (National Polytechnic Institute), Mexico City, Mexico

[1] Task-space (or Cartesian space) is defined by the position and orientation of the end effector of a robot. Joint-space is defined by angular displacements of each joint angle of a robot.

been used for robot control [7, 22] based on model-free and on-line algorithms [31]. RL methods for robot control include three subclasses: value iteration, policy iteration and policy search [8]. In value iteration subclass, the controller obtains the optimal policy by taking the minimal return from every value or action value function, e.g., Q-learning algorithm [38]. In policy iteration subclass, the controller evaluate their policies by constructing their value functions and use them to improve their policies, e.g., Sarsa [20], LSTD-Q algorithm. In policy search subclass, the controllers use optimization techniques to directly search for an optimal policy, e.g., policy gradient and actor-critic methods [12, 32]. Value and policy iteration algorithms require an expensive optimization procedure over the value function, meanwhile policy search algorithms store the value and the policy separately and the control actions can be calculated directly from the learned policy. Therefore policy gradient and actor-critic methods are preferred for robot control [17].

The above RL methods are designed for a single agent, i.e, one DOF. If the DOF of the robot is high, the space dimension and the computational load of the RL algorithm are increased [39, 40]. Multi-agent reinforcement learning (MARL) can deal with high DOFs [2], but it requires another kind of methodology [47] to deal with the high state-action space such as function approximators. The main problem of the MARL for robot control is the curse of dimensionality, which is caused by the exponential growth of the discrete state-action space [29]. To solve the dimensionality problem, several architectures are designed, such as policy gradient [12] and actor-critic methods [18], where the value function and the policy are represented by a differentiable parametrization [19]. However, these representations cannot be applied directly for redundant robots because the space dimension is still large [6]. So the control problems of redundant robots are:

1. For redundant robots, the accurate representation of the inverse kinematics solution of the Jacobian matrix are not available.
2. The inverse kinematics solution for redundant robots has multiple solutions.
3. The Jacobian matrix is not a square matrix and loses rank at singular points

In order to solve the above control problems of redundant robots, we do the following contributions in this paper:

1. We use the multi-agent and the actor-critic methods to modify RL. The proposed MARL can learn both inverse kinematic and velocity kinematic. The novel method does not need any function approximator [42, 43]. The curse of dimensionality is also avoided. Unlike the other Jacobian-based methods, our approach does not present controllability problem.
2. Real-time experiment comparisons with the Jacobian-based methods and neural networks are proposed by using an exoskeleton robot.

The outline of the paper is as follows: Sect. 2 provides the joint and task space control schemes for redundant robots using a velocity kinematics approach and neural networks; Sect. 3 shows the inverse kinematics solution using reinforcement learning and multi-agent reinforcement learning; the experiment results are given in Sects. 4 and 5 concludes the paper.

2 Redundant robot control in task-space

In joint-space, the dynamic of a robotic manipulator is of the form:

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau \tag{1}$$

where $M(q) \in \mathbb{R}^{n\times n}$ is a positive definite inertia matrix, $C(q,\dot{q}) \in \mathbb{R}^{n\times n}$ is the Coriolis matrix, $G(q) \in \mathbb{R}^n$ is the gravitational torques vector, $\tau \in \mathbb{R}^n$ is the driven torque and $q, \dot{q}, \ddot{q} \in \mathbb{R}^n$ are the position, velocity and acceleration of the joint angles. To design a task-space control law, we first need the relation between the joint angles $q \in \mathbb{R}^n$ and the end-effector $x \in \mathbb{R}^m$,

$$x = f(q), \quad q = f^{-1}(x) \tag{2}$$

where $f(\cdot) : \mathbb{R}^n \to \mathbb{R}^m$ is the forward kinematics, $f^{-1}(\cdot) : \mathbb{R}^m \to \mathbb{R}^n$ the inverse kinematics. However, for redundant robots the inverse kinematics solution $f^{-1}(\cdot)$ is not feasible because we have more joint space DOF than task space DOF, $n > m$,. To avoid this problem, the velocity kinematics is applied. It is expressed as

$$\dot{x} = \frac{\partial f(q)}{\partial q}\dot{q} = J(q)\dot{q} \tag{3}$$

where $J(q) \in \mathbb{R}^{n\times m}$ is the Jacobian matrix.

The Jacobian matrix has two associated subspaces: the range and the null space. The range of the Jacobian is given by the transformation mapping $\mathfrak{R}[J(q)] = \{J(q)\dot{q} | \dot{q} \in \mathbb{R}^n\}$; on the other hand, the null space of the Jacobian is given by the following transformation mapping $\mathfrak{N}[J(q)] = \{\dot{q} \in \mathbb{R}^n | J(q)\dot{q} = 0\}$. Redundant robots have null space, because there are some joint velocities in the null space $\mathfrak{N}[J(q)]$ that do not generate any velocity at the end effector, however they generate joint motions.

There are two methods for task-space robot control: task-space design and joint-space design, see Fig. 1. The

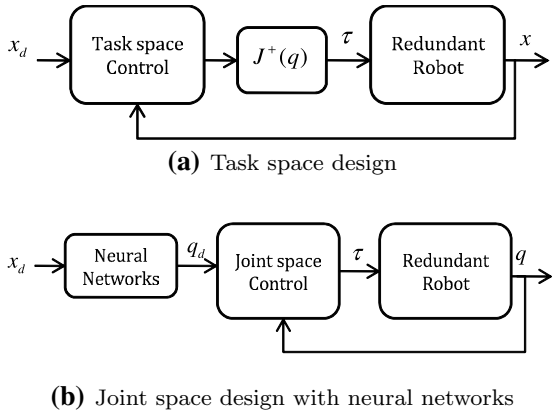


Fig. 1 Task space control methods of redundant robots

task-space design method needs the inverse of the Jacobian J^+. In this paper a PID controller [30] is used,

$$\tau = J^+\left(K_p e - K_d \dot{x} + K_i \int_0^t e(s)ds\right) \tag{4}$$

where $K_p, K_d, K_i \in \mathbb{R}^{m\times m}$ are the proportional, derivative and integral positive definite diagonal matrices gains, $e = x_d - x$ is the Cartesian position error.

The joint-space design method needs the inverse kinematics f^{-1}. We use neural networks to approximate the inverse kinematics solution. The joint-space PID control law is

$$\tau = K_p e_q - K_d \dot{q} + K_i \int_0^t e_q(s)ds \tag{5}$$

where $e_q = q_d(x_d) - q$ is the joint position error, $q_d(x_d)$ is obtained by the inverse kinematics, which is approximated by the neural networks. Here $K_p, K_d, K_i \in \mathbb{R}^{n\times n}$.

2.1 Task-space design

For redundant robots, the Jacobian matrix is not square. The simplest method to obtain the inverse of the Jacobian is to use the Moore–Penrose pseudo-inverse as

$$J^+ = J^\top \left(JJ^\top\right)^{-1}. \tag{6}$$

If the Jacobian J loses rank, the singular-value-decomposition (SVD) can approximate the pseudo-inverse as

$$J^+ = v\sigma^* w^\top \tag{7}$$

where σ^*, v and w are the singular decomposition matrices. The main problem of these pseudo-inverse methods is that they do not exploit the velocities in the null space. So they require additional tasks [26], which are specified as

$$\dot{q} = \dot{q}_{\mathfrak{N}} + \dot{q}_{\mathfrak{N}}$$

where the velocity in the null space is obtained by $\dot{q}_{\mathfrak{N}} = (I - J^+J)\upsilon$, for an arbitrary $\upsilon \in \mathbb{R}^n$ [24].

In the augmented Jacobian method [3], it is defined additional tasks. The additional task is denoted by $x_a = l(q)$, here $x_a \in \mathbb{R}^{n-m}$ is a vector defined by a function $l(\cdot)$ that depends on the joint angles. The augmented task-space is expressed as $z = [x^\top, x_a^\top]^\top$, so the new Jacobian

$$\dot{z} = \begin{bmatrix} J \\ L \end{bmatrix} \dot{q} = J_a \dot{q}, \quad L = \frac{\partial l(q)}{\partial q} \in \mathbb{R}^{(n-m)\times n} \tag{8}$$

becomes a square matrix, $J_a \in \mathbb{R}^{n\times n}$, and $\dot{q} = J_a^{-1}\dot{z}$. However, the augmented task-space may bring extra singularity, and J_a^{-1} may exist at all time.

To avoid the above problems, we use the following optimization method. We want to minimize the following cost function:

$$\mathcal{J} = \left\|J\dot{q} - \dot{x}_d\right\|^2 \tag{9}$$

where $\dot{x}_d$ is the desired position. To avoid singularities, the weighted norm of the joint rates is added to the above cost function as:

$$\mathcal{J} = \left\|J\dot{q} - \dot{x}_d\right\|^2 + \|\xi\dot{q}\|^2 \tag{10}$$

where ξ is a penalization constant. The solution of the optimization problem (10) is:

$$\dot{q} = (J^\top J + \xi^2 I)^{-1} J^\top \dot{x}_d \tag{11}$$

The above solution is unique and close to the exact solution, while it avoids infinite high joint rates due to the singularities.

Remark 1 The pseudo-inverse method does not exploit the velocities in the null space. The SVD method is an off-line method which loses rank in presence of singularities. The augmented Jacobian incorporates new kinematics, which arises to new singularities that we do not have before. Our optimal method can deal with some kinematic problems by choosing some ξ. However small values of ξ at a singularity point gives big rate velocities, big values of ξ yields small rate velocities.

2.2 Joint-space design

In joint-space we use neural networks to learn the inverse kinematics. The NN with L hidden nodes is defined by

$$q_i(x) = \sum_{j=1}^{L} W_j G(a_j, b_j, x) \tag{12}$$

where the output $q_i(x)$ is the joint angle, $i = 1, 2, \ldots, n$ is tghe joint index, the input $x \in \mathbb{R}^6$ denote the robot end-effector position (X, Y, Z) and orientation (α, β, γ), a_j and b_j are the learning parameters of the hidden nodes , W_j is the weight. The activation function $g(x)$ is an hyperbolic tangent sigmoid function, then

$$G(a_j, b_j, x) = g(a_j \cdot x + b_j). \tag{13}$$

Given the training samples $x_s \subset \mathbb{R}^n \times \mathbb{R}^m$, $s = 1, 2, \ldots, N$, the output is expected to be equal to the targets as

$$q_i(x) = \sum_{j=1}^{L} W_j G(a_j, b_j, x_s) \tag{14}$$

In matrix form (14) is

$$HW = T \tag{15}$$

where $H = \begin{bmatrix} G(a_1, b_1, x_1), & \cdots, & G(a_L, b_L, x_1) \\ \vdots, & \cdots, & \vdots \\ G(a_1, b_1, x_N), & \cdots, & G(a_L, b_L, x_N) \end{bmatrix}_{N \times L}$, $W = \begin{bmatrix} W_1^\top \\ \vdots \\ W_L^\top \end{bmatrix}_{L \times m}$, $T = \begin{bmatrix} t_1^\top \\ \vdots \\ t_N^\top \end{bmatrix}_{N \times m}$. By the least square method, the minimizations of $\|HW - T\|$ yields the following Moore–Penrose pseudo-inverse solution

$$\hat{W} = H^+ T \tag{16}$$

Remark 2 The joint-space controller (5) requires that the NN gives a good approximation of the inverse kinematics solution, such that $q_d(x_d)$ is reliable. A wrong inverse kinematics solution may lead to wrong joint angles $q_d(x_d)$. To obtain a good solution, NN needs big training data and a long time learning phase. The PID gains do not need the information of the inverse kinematics.

3 Multi-agent reinforcement learning for redundant robot control

Reinforcement learning (RL) for robot control can be regarded as a Markov decision process (MDP) [38]. The MDP is a tuple $\langle \mathcal{X}, \Delta Q, f, \rho \rangle$, where $\mathcal{X}$ is the state space of the task space components, ΔQ is the action space that is represented by small joint displacements, $f : \mathcal{X} \times \Delta Q \to \mathcal{X}$ is the state transition function determined by the forward kinematics and $\rho : \mathcal{X} \times \Delta Q \to \mathbb{R}$ is a scalar reward function.

The forward kinematics $f : \mathcal{X} \times \Delta Q \to \mathcal{X}$, returns the new state $x_{t+1} \in \mathcal{X}$ of the task-space components after applying a small joint displacement $\Delta q \in \Delta Q$ at the current joint state q. After the transition, the RL algorithm receives a scalar reward $\rho(x_t, q_t + \Delta q_t)$ which evaluates the immediate effect of the displacement Δq, namely the transition from x_t to x_{t+1}. The joint displacements are chosen according to the policy $q(x) : \mathcal{X} \to \Delta Q$.

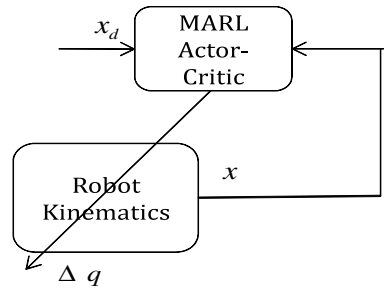


Fig. 2 RL/MARL control scheme

The proposed learning scheme using RL is shown in Fig. 2. Here the control actions are joint displacement $\Delta q = [\Delta q_1, \ldots, \Delta q_r]^\top$, where Δq_i, $i = 1, \ldots, r$, r is the unmber of the discrete displacements. The RL algorithm uses the joint space displacements to update the robot kinematics, until the robot achieves the desired position x_d.

The updates of the robot pose are given by the forward kinematics (2) as

$$x = f(q + \Delta q) = f(\sigma) \tag{17}$$

where $\sigma = q + \Delta q$. The objective of our RL is to find the inverse kinematics solution and force the end-effector to reach certain desired position in task-space, such that the discounted sum of future rewards is minimized. Let define the value function as

$$V^{q(x)}(x) = \sum_{i=t}^{\infty} \gamma^{i-t} \rho(x, \sigma) \ \text{ with } x_t = x \tag{18}$$

where $\gamma \in (0, 1]$ is the discount factor, $\rho(x, \sigma)$ is the reward function of the control object, for example the quadratic form of position tracking error. (18) satisfies the following Bellman equation

$$V^{q(x)}(x) = \rho(x, q(x)) + \gamma V^{q(x)}(x_{t+1}) \tag{19}$$

where $x_{t+1} = f(x_t, q(x_t))$ is the forward kinematics of (17). The optimal value function satisfies

$$V^*(x) = \min_{q(x)} V^{q(x)}(x)$$

The inverse kinematics solution is given by

$$q^*(x) = \arg\min_{\sigma} \left[\rho(x,\sigma) + \gamma V^*(x_{t+1})\right] \tag{20}$$

The value function can be written as the action value function by the following Bellman equation

$$Q^{q(x)}(x,\sigma) = \rho(x,\sigma) + \gamma Q^{q(x)}(x_{t+1}, q(x_{t+1})) \tag{21}$$

The optimal action value function satisfies

$$Q^*(x,\sigma) = \min_{q(x)} Q^{q(x)}(x,\sigma)$$

Finally the inverse kinematics solution is

$$q^*(x) = \arg\min_{\sigma} Q^*(x,\sigma) \tag{22}$$

In (20) the Q-function depends on the joint displacements, while V-function describes the task-space states. It is easier to use V-functions than Q-functions, since the Q-functions depend both on x and Δq, and need the forward kinematics transition [23]. The Bellman equations (19) and (21) are the baseline equations to design RL algorithms, and in this form, they are known as tabular methods for a single DOF (also called agent) [8]. Since redundant robots have many joint DOF, then the single DOF RL cannot be applied because there are too many tables to store all possible combinations of the joint angles. Also the desired position x_d components are coupled, therefore we cannot use separate reward functions for each DOF for the task-space control design.

In this paper, we use a multi-agent reinforcement learning (MARL) [22, 48]. Now the MDP is a stochastic game, which is a tuple

$$\langle \mathcal{X}, \Delta Q_1, \dots, \Delta Q_n, f, \rho_1, \dots, \rho_n \rangle$$

where n is the number of the joints, $\Delta Q_i, i = 1, \dots, n$ are the action space (joint displacements space) of each joint angle, $\rho_i : \mathcal{X} \times \Delta Q \to \mathbb{R}, i = 1, \dots, n$, are the reward functions of the joint angles.

The complete joint displacements space is denoted by $\Delta Q = \Delta Q_1 \times \cdots \times \Delta Q_n$. The joint displacements are $\Delta q = \left[\Delta q_{i,t}^\top, \dots, \Delta q_{n,t}^\top\right]^\top \in \Delta Q, \Delta q_{i,t} \in \Delta Q_i$. The policies of the inverse kinematics solution are

$$q_i : \mathcal{X} \to \Delta Q_i \in q(x)$$

Also the joint updates are denoted as

$$\sigma = \left[\sigma_1, \cdots, \sigma_n\right]^\top$$

where $\sigma_i = q_i + \Delta q_i$.

In a fully cooperative stochastic game [39], the reward functions are the same for all the joint DOFs, i.e., $\rho_1 = \cdots = \rho_n$. For robot control, the reinforcement learning algorithm is modelled as a multi-agent case [22] where for a joint-space control, it is used different rewards for each joint angle, on the other hand, in a task space control it is modelled as a fully cooperative stochastic game, which is a more challenging issue. Our MARL is a fully cooperative stochastic game, then the inverse kinematics solution (20) and (22) are rewritten as:

$$\begin{aligned} q_i^*(x) &= \arg\min_{\sigma_i} \min_{\sigma_1,\dots,\sigma_{i+1},\dots,\sigma_n} [\rho(x,\sigma) + \gamma V^*(x_{t+1})] \\ q_i^*(x) &= \arg\min_{\sigma_i} \min_{\sigma_1,\dots,\sigma_{i+1},\dots,\sigma_n} Q^*(x,\sigma) \end{aligned} \tag{23}$$

The main problem of the MARL is the curse of dimensionality [29], which is caused by the exponential growth of the discrete state-action space. There are many algorithms that deal with the curse of dimensionality such as probabilistic inference for learning control (PILCO) [11], proximal policy optimization (PPO) [34], deterministic policy gradients (DPG) [35], k-nearest neighbors [32], among others. These algorithms use different approximators in order to reduce the computational complexity.

However, they have to use large state-action spaces to obtain good solutions for the MARL, because even a small approximation error may cause wrong trajectories for the robot [28]. In order to avoid the function approximation and large state-action space, in this paper we use the actor-critic method and the kinematic learning (17) for the MARL.

Our actor-critic method only needs small displacements to update the forward kinematics, see Fig. 2. We avoid to use many joint displacements, because the kinematic learning algorithm updates the joint angles in each iteration ΔQ_i.

The proposed actor-critic method only needs three hyperparameters: two learning rates for the critic and the actor, and one discount factor. Compared with the other approximator-based MARL methods, our method simplify the implementation of the MARL for the redundant robot.

We use two value functions and the actor-critic method to obtain the inverse kinematics solution of each joint without any function approximator. The value function-based actor-critic method uses the state value function $V(\cdot)$ as the critic. The critic is updated by

$$V_{t+1}^{q(x)}(x_t) = V_t^{q(x)}(x_t) + \alpha_c \left(\rho(x,\sigma) + \gamma V_t^{q(x)}(x_{t+1}) - V_t^{q(x)}(x_t)\right) \tag{24}$$

where $\alpha_c > 0$ is the critic learning rate.

The action value function $Q(\cdot)$ is used to obtain the new control policy, and the inverse kinematics. The actor is updated by

$$Q_{t+1}^{q(x)}(x_t,\sigma_t) = Q_t^{q(x)}(x_t,\sigma_t) + \alpha_a \delta_t \tag{25}$$

where $\alpha_a > 0$ is the actor learning rate. We use the following TD-error to update the Q−function,

$$\delta_t = \rho(x,\sigma) + \gamma V_t^{q(x)}(x_{t+1}) - V_t^{q(x)}(x_t). \tag{26}$$

The V-function learns faster than a Q function causing fast bootstrapping of the inverse kinematic solution $q(x)$. Here we use the actor-critic method, the actor is updated by the temporal difference rule of the critic, which has fast bootstrapping of the inverse kinematics solution.

The inverse kinematics solution can be greedy obtained by the next expression

$$q^*_{i,t} = \arg\min_{\sigma_i} \min_{\sigma_1,\ldots,\sigma_{i+1},\ldots,\sigma_n} Q^*_t(x_t, \sigma).$$

The algorithm of the value function-based actor critic is given by the next algorithm.

1. **Algorithm 1**.
2. Initialize discount factor γ, and actor-critic learning rates α_c, α_a, and desired position x_d
3. Repeat {for each episode}
   - Initialize x_0
   - Repeat {for each step of episode $t = 0, 1, \ldots$ and joint $i = 1, \ldots, n$}
   - Take displacements $\Delta q_{i,t}$, update $\sigma_{i,t} = q_{i,t} + \Delta q_{i,t}$ and measure $\rho(x, \sigma)$ and x_{t+1}
   - Update the critic by (24)
   - Update the actor by (25)
   - Choose $\Delta q_{i,t+1}$ from x_{t+1} greedily using any exploration technique
   - $x_t \leftarrow x_{t+1}, \sigma_{i,t} \leftarrow \sigma_{i,t+1}$
4. x_t is terminal learning
5. ends

Remark 3 The main problem of classical RL for robot control is the curse of dimensionality. When the robot is redundant, the learning space are huge. Our MARL method uses a kinematic learning to avoid large action-state space. We use small discrete action space for updating, which includes small robot joint positions. In task-space, the input space of RL algorithm is composed of the end-effector position and orientation. The main advantage of our MARL is that the inverse kinematics is obtained on-line without any function approximator and it is achieved in less learning time.

The convergence proof of the two value function-based actor-critic algorithm for the inverse kinematics learning is given in the following theorem.

Lemma 1 [21] *Let P_t be a sequence of increasing σ-fields, such that ζ_0 and Δ_0 are P_0-measurable and ζ_t, Δ_t and F_{t-1} are P_t-measurable, $t = 1, 2, \ldots$. For any random iterative process (ζ_t, Δ_t, F_t), $t \geq 0$, where $\zeta_t, \Delta_t, F_t : \mathcal{X} \rightarrow \mathbb{R}$ satisfy the equation*

$$\Delta_{t+1}(z_t) = (1 - \zeta_t(z_t))\Delta_t(z_t) + \zeta_t(z_t)F_t(z_t)$$

where $z_t \in Z$. Assume that the following hold: (1) $\zeta_t(z_t) \in [0, 1], \sum_t \zeta_t = \infty, \sum_t \zeta_t^2 < \infty$ *w.p.1 and* $\forall z \neq z_t : \zeta_t(z) = 0$. (2) $\|E\{F_t|P_t\}\| \leq \kappa\|\Delta_t\| + c_t$, *where $\kappa \in [0, 1)$ and c_t converges to zero w.p.1.,* (3) $Var\{F_t(z_t)|P_t\} \leq K(1 + \kappa\|\Delta_t\|)^2$, *$K$ is some positive constant, $\|\cdot\|$ is the infinity norm. Then Δ_t converges to zero with probability one.*

Theorem 1 *Consider that each state-action pair is visited an infinite number of times and that the following conditions are fulfilled:* (1) $\gamma \in [0, 1)$, (2) $\alpha_c, \alpha_a \in [0, 1]$, $\sum_t \alpha_c = \infty, \sum_t \alpha_a = \infty, \sum_t {\alpha_c}^2 < \infty, \sum_t {\alpha_a}^2 < \infty$ *w.p.1.*)3) $Var\{R^{x'}_{x\mathbf{u}}\} < \infty$, *where $R^{x'}_{x\mathbf{u}}$ is the expected immediate cost, Q-function and V-function are updated as in Algorithm 1, then they converge to the optimal value functions V^* and Q^* respectively, with probability one.*

Proof Algorithm 1 uses two separate value functions. We first prove the convergence of the actor part. For any fixed policy, the value function can be written in the form of action value function as follows

$$V(x_t) = Q(x_t, \sigma_t), \quad V(x_{t+1}) = Q(x_{t+1}, \sigma_{t+1})$$

This means for any fixed control policy $q(x)$, the state value function V is equal to the state-action value function Q. Applying Lemma 1 with $\zeta = \alpha_a$,, $\Delta_t = Q_t - Q^*_t$, and $Z = \mathcal{X} \times \Delta Q$, $P_t = \{Q_0, x_0, \Delta q_0, \alpha_{a,0}, \rho, x_1, \ldots, x_t, \Delta q_t\}$, P_t is an increasing σ-field. And

$$F_t(x_t, \sigma_t) = \rho(x, \sigma) + \gamma\left[Q_t(x_{t+1}, \sigma_{t+1}) \pm \min_\sigma Q_t(x_{t+1}, \sigma)\right] - Q^*_t(x_t, \sigma_t)$$

The first and third condition of Lemma 1 hold by the Theorem conditions, therefore we only require to prove the second condition about the contraction of F_t. We can write F_t as:

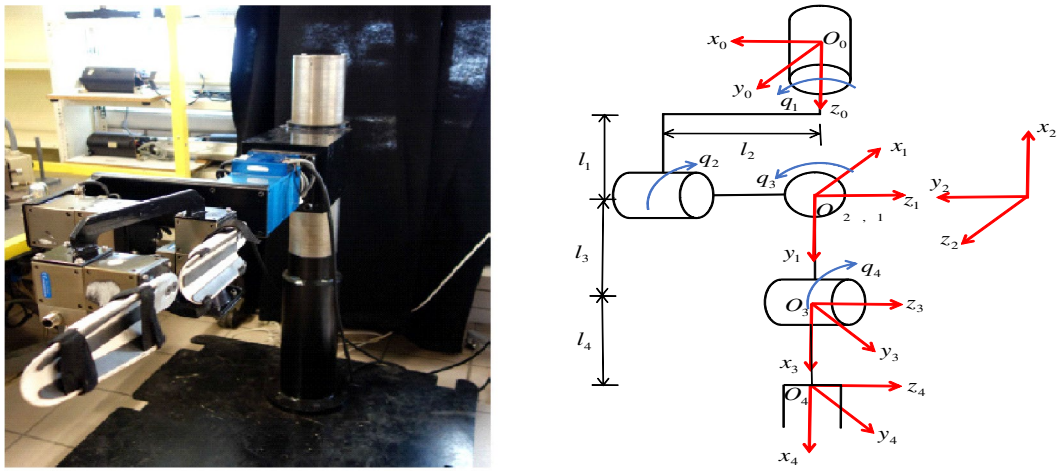
$$F_t(x_t, \sigma_t) = G_t + \gamma\left(Q_t(x_{t+1}, \sigma_{t+1}) - \min_\sigma Q_t(x_{t+1}, \sigma)\right)$$

where $G_t = r_{t+1} + \gamma \min_\sigma Q_t(x_{t+1}, \sigma) - Q^*_t(x_t, \sigma_t)$, which stands to the value of F_t of the standard Q-learning algorithm [29]. It is well known that $E\{G_t|P_t\} \leq \gamma\|\Delta_t\|$, and from Lemma 1 we can identify that

$$c_t = \gamma(Q_t(x_{t+1}, \sigma_{t+1}) - \min_\sigma Q_t(x_{t+1}, \sigma))$$

and it is sufficient to prove that the term in brackets of c_t converges to zero. Suppose that $Q_2(x_{t+1}, a) = \min_\sigma Q_t(x_{t+1}, \sigma)$ where a are the actions that minimize Q_t. The update for $\Delta^a_t = Q_t - Q_2$ is:

$$\Delta^a_{t+1} = (1 - \alpha_a)\Delta^a_t + \alpha_a c_t$$



(a) 4-DOF Robot **(b)** D.H. frames

Fig. 3 4-DOF Exoskeleton

Assume that the inequality $E\{Q_t(x_{t+1},\sigma_{t+1})|P_t\} \le E\{Q_2(x_{t+1},a)|P_t\}$ is satisfied, therefore

$$\begin{aligned}\left|E\{c_t|P_t\}\right| &= \gamma E\{Q_t(x_{t+1},\sigma_{t+1}) - Q_2(x_{t+1},a)|P_t\}\\ &\le \gamma E\{Q_t(x_{t+1},a) - Q_2(x_{t+1},a)|P_t\} \le \gamma\left\|\Delta_t^a\right\|\end{aligned}$$

Now assume that the following inequality is satisfied $E\{Q_2(x_{t+1},a)|P_t\} \le E\{Q_t(x_{t+1},\sigma_{t+1})|P_{t\}}$, therefore

$$\begin{aligned}\left|E\{c_t|P_t\}\right| &= \gamma E\{Q_t(x_{t+1},\sigma_{t+1}) - Q_2(x_{t+1},a)|P_t\}\\ &\le \gamma E\{Q_t(x_{t+1},\sigma_{t+1}) - Q_2(x_{t+1},\sigma_{t+1})|P_t\}\\ &\le \gamma\left\|\Delta_t^a\right\|\end{aligned}$$

Then applying Lemma 1 yields the convergence to zero w.p.1 of Δ_t^a and the convergence of Q_t to Q^*. The critic is a well-known TD update rule, whose convergence is similar to the actor convergence presented above, therefore V converges to V^* on the limit.

Remark 4 In this paper the actor and critic learning rates α_a and α_c are chosen as constant values, hence they do not satisfy the condition $\sum_t \alpha_t^2 < \infty$. However we use an ε -greedy exploration strategy [31] such that the state-action space is infinitely visited. Thus fulfilling the conditions of Theorem 1.

4 Experiment results

In this application, we use a 4-DOF exoskeleton robot shown in Fig. 3. The control objective of the robot is to move the end-effector to desired position. So the dimension in task-space is three, while the dimension in the joint-space is four. It is a redundant robot. We will compare our MARL with the following classical methods: the Jacobian-based augmented method (J^+ AJ), the Jacobian-based singularity avoidance (J^+ SA), the Jacobian-based singular-value-decomposition (J^+ SVD), and multilayer neural networks (SLFNN). The real time environment is Simulink and Matlab 2012. The communication is the CAN protocol, which enables the PC to communicate with the actuators.

The Denavit Hartenberg parameters of Fig. 3 are given in Table 1.

So the forward kinematics of the robot is:

$$\begin{aligned}X &= c_1(c_2c_3(l_3+l_4c_4) - l_4s_2s_4) - (l_3+l_4c_4)s_1s_3\\ Y &= c_2c_3(l_3+l_4c_4)s_1 - l_4s_1s_2s_4 + c_1(l_3+l_4c_4)s_3\\ Z &= l_1 + c_3(l_3+l_4c_4)s_2 + l_4c_2s_4\end{aligned} \tag{27}$$

where $c_i = \cos(q_i), s_i = \sin(q_i), i = 1,2,3,4$, $l_1 = 0.228, l_3 = l_4 = 0.22$ meters. By differentiating (27) with respect to the joint angles, the analytic Jacobian $J(q)$ is obtained. The range of the analytic Jacobian is $\Re[J(q)] = 3$.

We first consider the singularity case. We set the initial joint positions as $q_0 = \left[0, \frac{\pi}{2}, 0, 0\right]^\top$ which is a singular position. The pseudo-inverse method can be applied. However, the term $\det(J^\top J) = 0$ for all joint positions, hence (6) cannot be applied. We use SVD to compute the pseudo-inverse of

Table 1 Denavit Hartenberg parameters

Joint i	θ_i	d_i	a_i	α_i
1	q_1	l_1	0	$\frac{\pi}{2}$
2	q_2	0	0	$-\frac{\pi}{2}$
3	q_3	0	l_3	$\frac{\pi}{2}$
4	q_4	0	l_4	0

Table 2 PID control gains

Gain	SVD	AJ	SA	SLFNN
K_p	$20I$	$30I$	$20I$	$9I$
K_d	$12I$	$20I$	$12I$	$4I$
K_i	$8I$	$14I$	$8I$	$2I$

the Jacobian at the initial position as $J^+ = \begin{bmatrix} 0 & 0 & 0 \\ -1.8182 & 0 & 0 \\ 0 & 2.2727 & 0 \\ -0.9091 & 0 & 0 \end{bmatrix}$. Because the last column is zero, J^+ cannot give correct mapping between task-space and joint-space. The augmented Jacobian method adds an angular velocity Jacobian component as the additional task to obtain a square Jacobian. It is added the last row of the angular velocity Jacobian [27, 45], i.e., $[1, 0, c2, s2s3]^\top$. The range of the augmented Jacobian is $\Re(J_a(q)) = 4$. Since the augmented Jacobian is singular at the initial pose, we use the SVD method. The inverse of the augmented Jacobian is $J_a^{-1} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ -1.8182 & 0 & 0 & 0 \\ 0 & 2.2727 & 0 & 0 \\ -0.9091 & 0 & 0 & 0 \end{bmatrix}$. To avoid the singularity, we use the penalization constant $\xi = 0.1$ in (10).

The training set for the SLFNN is obtained using a homogeneous discretization of the joint-space and the forward kinematics. The discretized area of the workspace for the 4-DOF exoskeleton is proposed as the human shoulder and elbow movement,

$$\frac{7\pi}{18} \le q_1 \le \frac{17\pi}{36}, \quad -\frac{11\pi}{36} \le q_2 \le \pi$$
$$0 \le q_3 \le \frac{2\pi}{3}, \quad 0 \le q_4 \le \frac{29\pi}{36}$$

We have 642,096 data points, 70% of the available data are for training. There are 100 neurons at the hidden layer.

The learning parameters of the MARL are $\alpha_a = 0.1$, $\alpha_c = 0.3$ and $\gamma = 0.9$. For the ε-greedy exploration strategy it is used a probability of $\varepsilon = 0.01$. The MARL runs for 1, 000 episodes with 1, 000 steps per episode. The desired position is $x_d = [-0.24, -0.196, 0.207]^\top$. The reward is designed as the quadratic error between the desired reference and the robot pose:

$$\rho(x, \sigma) = \|x_d - x\|^2 = \|x_d - f(\sigma)\|^2 \tag{28}$$

The PID gains are given in Table 2, where I stands to the identity matrix of dimension $m \times m$ for the SVD, AJ ans SA methods; and of $n \times n$ for the SLFNN method.

The results at the Z axis are given in Fig. 4. The pseudo-inverse methods using the SVD and the AJ show that they cannot achieve the control task because the pseudo-inverse matrix loses a degree of freedom. SA method requires knowledge of the robot kinematics. The Jacobian-based methods produce high joint rates when the robot is at a singular point, so the parameter ξ must be chosen carefully.

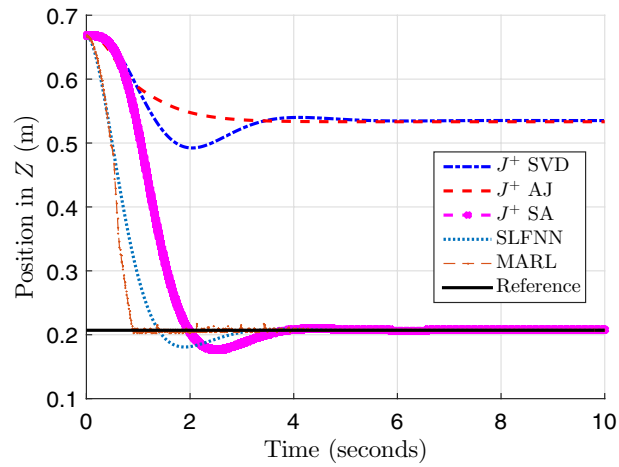


Fig. 4 Position in Z

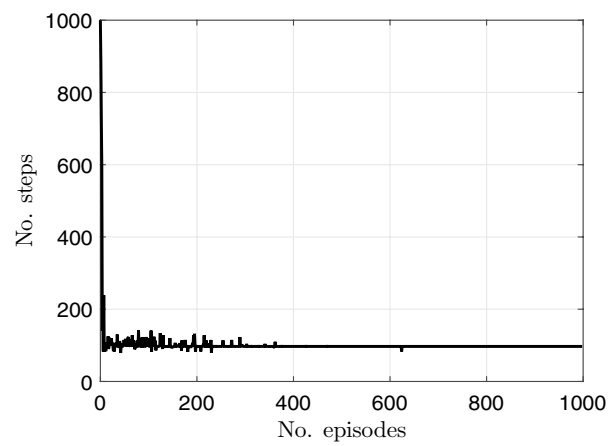


Fig. 5 MARL learning curve

The SLFNN requires off-line training, and it takes about 6 h due to the complexity of the neural networks. The MARL algorithm converges to the desired reference faster than the other methods. The algorithm achieves the control task without controllability problems. The learning curve in Fig. 5 shows that the algorithm converges rapidly since the critic helps the actor to learn the optimal solution in less learning time. Figure 6 shows how our MARL minimizes the total cumulative reward in each learning episode, i.e., $\sum_{i=1}^{T} \rho_i(x, \sigma)$. The reward decreases, because it is a minimization problem.

Now we modify the desired position and compare our MARL with J^+ SA and SLFNN methods. The desired position is $x_d = [-0.086, 0.138, 0.62]^\top$, the initial joint position is the same as the singular case. We use the same learning parameters. The PID gains in joint space are modified as

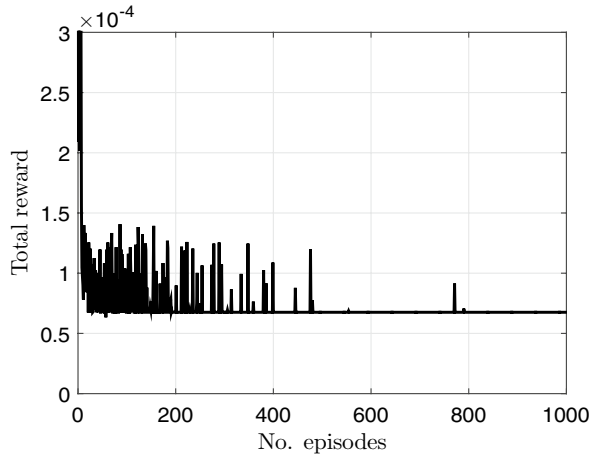


Fig. 6 Total reward curve

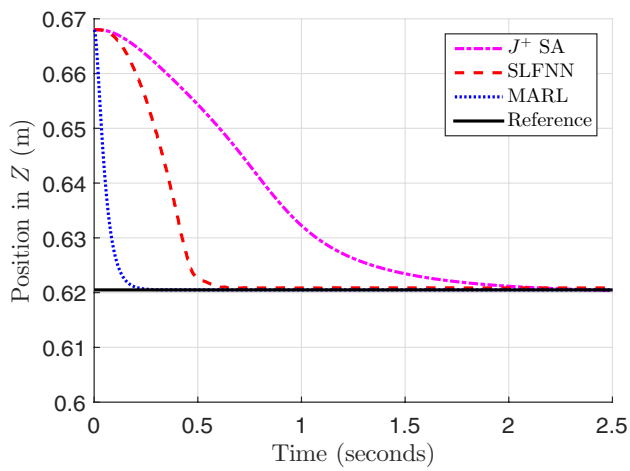


Fig. 7 Position in *Z*. Experiment results

$K_p = 90I$, $K_d = 2I$, and $K_i = 5I$. In task-space, $K_p = 180I$, $K_d = 4I$ and $K_i = 10I$, where I is the identity matrix with an appropriate dimension. The comparison results are given in Fig. 7.

The above results show the effectiveness of our approach. The J^+ SA method depends on the controller gains and the penalization constant which determines how fast is the controller response. The MARL is a model free algorithm and obtains a near-optimal solution for a robot position control for any pose using by using its kinematic learning method. The advantages of the kinematic learning method is that we do not care about the robot dynamics and possible uncertainties and disturbances, instead we only require the kinematic model to achieve the control task. Also the MARL converges faster than the SLFNN and can be applied on-line.

When the MARL converges, the inverse kinematic for the desired task is obtained. The inverse kinematics can be used for the repetitive tasks, and to transform a task space control problem to a joint-space control problem in an easy way.

5 Conclusion

In this work a novel multi-agent reinforcement learning is applied for redundant robot control. This new method does not need of the knowledge inverse kinematics, Jacobian matrix and the use of function approximators. The novel reinforcement learning uses a two value functions actor-critic algorithm and a kinematic learning approach to obtain on-line the inverse kinematics solution without controllability problems. Also the curse of dimensionality is overcome. The convergence of the proposed algorithm is proven. The experiment uses a 4-DOF exoskeleton robot. We verify our approach with standard Jacobian-based and neural networks methods. In the further, we will combine the fuzzy control [36, 37] and MARL for the redundant robot control.

References

1. Ahmadi S, Fateh M (2018) Task-space asymptotic tracking control of robots using a direct adaptive Taylor series controller. J Vib Control 24(23):5570–5584. https://doi.org/10.1177/1077546318758800
2. Ansari Y, Falotico E (2016) A multiagent reinforcement learning approach for inverse kinematics oh high dimensional manipulators with precision positioning. In: 6th IEEE RAS/EMBS international conference on biomedical robotics and biomechatronics (BioRob). https://doi.org/10.1109/BIOROB.2016.7523669
3. Atashzar S, Tavakoli M, Patel R (2018) A computational-model-based study of supervised haptics-enabled therapist-in-the-loop training for upper-limb poststroke robotic rehabilitation. IEEE/ASME Trans Mechatron 23(2):562–574. https://doi.org/10.1109/TMECH.2018.2806918
4. Axinte D, Dong X, Palmer D, Rushworth A, Guzman S, Olarra A (2018) Miror-miniaturized robotic systems for holisticin-siture-pair and maintenance works in restrained and hazardous environments. IEEE/ASME Trans Mechatron 23(2):978–981. https://doi.org/10.1109/TMECH.2018.2800285
5. Bcsi B, Nguyen-Tuong D, Csat L, Schlkopf B, Peters J (2011) Learning inverse kinematics with structured prediction. IEEE/RSJ Int Conf Intell Robots Syst. https://doi.org/10.1109/IROS.2011.6094666
6. Bitzer S, Howard M, Vijayakumar S (2010) Using dimensionality reduction to exploit constraints in reinforcement learning. IEEE/RSJ Int Conf Intell Robots Syst (IROS). https://doi.org/10.1109/IROS.2010.5650243
7. Buşoniu L, Babûska R, De Schutter B (2010) Multi-agent reinforcement learning: an overview. In: Srinivasan D, Jain L (eds) Innovations in multi-agent systems and applications—1. Studies in computational intelligence. Lecture notes in computer science, vol 310. Springer, Berlin. https://doi.org/10.1007/978-3-642-14435-6_7
8. Buşoniu L, Babûska R, De Schutter B, Ernst D (2010) Reinforcement learning and dynamic programming using function

approximators. Automation and Control Engineering Series. CRC Press, Boca Raton
9. Cheah C, Li X (2011) Singularity-robust task-space tracking control of robot. IEEE Int Conf Robot Autom. https://doi.org/10.1109/ICRA.2011.5979932
10. Csistzar A, Eilers J, Verl A (2017) On solving the inverse kinematics problem using neural networks. In: 24th international conference on mechatronics and machine vision in practice. https://doi.org/10.1109/M2VIP.2017.8211457
11. Deisenroth M, Rasmussen C (2011) PILCO: A model-based and data-efficient approach to policy search. In: Proceedings of the 28th international conference on machine learning, Bellevue, WAA, USA
12. Deisenroth MP, Neumann G, Peters J (2011) A survey on policy search for robotics. Found Trends Robot 2(1–2):1–142. https://doi.org/10.1561/2300000021
13. Duka A (2014) Neural network based inverse kinematics solution for trajectory tracking of a robotic arm. In: Procedia technology, the 7th international conference interdisciplinarity in engineering, INTER-ENG 2013. Petru Maior University of Tirgu Mures, Romania. https://doi.org/10.1016/j.protcy.2013.12.451
14. Feng Y, Yao-nan W, Yi-min Y (2012) Inverse kinematics solution for robot manipulator based on neural network under joint subspace. Int J Comput Commun Control 7(3):459–472. https://doi.org/10.15837/ijccc.2012.3.1387
15. Galicki M (2016) Finite-time trajectory tracking control in task space of robotic manipulators. Automatica 67:165–170. https://doi.org/10.1016/j.automatica.2016.01.025
16. Galicki M (2016) Robust task space trajectory tracking control of robotic manipulators. Int J Appl Mech Eng 21(3):547–568. https://doi.org/10.1515/ijame-2016-0033
17. Grondman I, Buşoniu L, Babûska R (2012) Model learning actor-critic algorithms: performance evaluation in a motion control task. In: 51st IEEE conference on decision and control (CDC), pp 5272–5277. https://doi.org/10.1109/CDC.2012.6426427
18. Grondman I, Vaandrager M, Buşoniu L, Babûska R, Schuitema E (2011) Actor-critic control with reference model learning. In: Proceedings of the 18th World congress the international federation of automatic control, pp 14723–14728. https://doi.org/10.3182/20110828-6-IT-1002.00759
19. Grondman I, Vaandrager M, Buşoniu L, Babûska R, Schuitema E (2012a) Efficient model learning methods for actor-critic control. IEEE Trans Syst Man Cybern B Cybern 42(3):291–602. https://doi.org/10.1109/TSMCB.2011.2170565
20. Hyatt P (2019) Configuration estimation for accurate position control of large-scale soft robots. IEEE/ASME Trans Mechatron 24(1):88–99. https://doi.org/10.1109/TMECH.2018.2878228
21. Jaakola TMJ, Singh S (1994) On the convergence of stochastic iterative dyanamic programming algorithms. Neural Comput 6(6):1185–1201. https://doi.org/10.1162/neco.1994.6.6.1185
22. Kober J, Bagnell J, Peters J (2013) Reinforcement learning in robotics: a survey. Int J Robot Res 32(11):1238–1274. https://doi.org/10.1007/978-3-319-03194-1_2
23. Lewis F, Vrable D, Vamvoudakis K (2012) Reinforcement learning and feedback control: using natural decision methods to desgin optimal adaptive controllers. IEEE Control Syst Mag 32(6):76–105. https://doi.org/10.1109/MCS.2012.2214134
24. Luya L, Gruver W, Zhang Q, Yang Z (2001) Kinematic control of redundant robots and the motion optimizability measure. IEEE Trans Syst Man Cybern Part B Cybern 31(1):155–160. https://doi.org/10.1109/3477.907575
25. Moon Y, Seo J, Choi J (2015) Development of new end-effector for proof-of-concept of fully robotic multichannel biopsy. IEEE/ASME Trans Mechatron 20(6):2996–3008. https://doi.org/10.1109/TMECH.2015.2418793
26. Patel R, Shadpey F (2005) Control of redundant manipulators: theory and experiments. Springer, Berlin. https://doi.org/10.1007/b93979
27. Perrusquía A, Yu W (2020) Human-in-the-loop control using euler angles. J Intell Robot Syst 97:271–285. https://doi.org/10.1007/s10846-019-01058-2
28. Perrusquía A, Yu W (2020) Robot position/force control in unknown environment using hybrid reinforcement learning. Cybern Syst. https://doi.org/10.1080/01969722.2020.1758466
29. Perrusquía A, Yu W, Soria A (2019) Large space dimension reinforcement learning for robot position/force discrete control. In: 2019 6th international conference on control, decision and information technologies (CoDIT 2019), Paris, France. https://doi.org/10.1109/CoDIT.2019.8820575
30. Perrusquía A, Yu W, Soria A (2019) Optimal contact force in unknown environments using reinforcement learning and model-free controllers. In: 16th international conference on electrical engineering, computing science and automatic control (CCE), Mexico city, Mexico. https://doi.org/10.1109/ICEEE.2019.8884518
31. Perrusquía A, Yu W, Soria A (2019) Position/force control of robots manipulators using reinforcement learning. Ind Robot Int J Robot Res Appl 46(2):267–280. https://doi.org/10.1108/IR-10-2018-0209
32. Perrusquiía A, Yu W (2020) Robust control under worst-case uncertainty for unknown nonlinear systems using modified reinforcement learning. Int J Robust Nonlinear Control 30(7):2920–2936. https://doi.org/10.1002/rnc.4911
33. Rolf M, Steil J (2014) Efficient exploratory learning of inverse kinematics on a bionic elephant trunk. IEEE Trans Neural Netw Learn Syst 25(6):1147–1160. https://doi.org/10.1109/TNNLS.2013.2287890
34. Schulman J, Wolski F, Klimov O (2017) Proximal policy optimization algorithms. arXiv:1707.06347
35. Silver D, Lever G, Hess N, Degris T, Wierstra D, Riedmiller M (2014) Deterministic policy gradient algorithms. In: Proceedings of the 31st international conference on machine learning, Beijing, China, vol 32, pp 387–395
36. Sun K, Liu L, Qiu J, Feng G (2020) Fuzzy adaptive finite-time fault tolerant control for strict-feedback nonlinear systems. IEEE Trans Fuzzy Syst. https://doi.org/10.1109/TFUZZ.2020.2965890
37. Sun K, Qiu J, Karimi H, Fu Y (2020) Event- triggered robust fuzzy adaptive finite-time control of nonlinear systems with prescribed performance. IEEE Trans Fuzzy Syst. https://doi.org/10.1109/TFUZZ.2020.2979129
38. Sutton RAB (1998) Reinforcement learning: an introduction. MIT Press, Cambridge
39. Tamei T, Shibata T (2009) Policy gradient learning of cooperative interaction with a robot using user's biological signals. Int Conf Neural Inf Process (ICONIP). https://doi.org/10.1007/978-3-642-03040-6_125
40. Theodorou E, Buchli J, Schaal S (2010) Reinforcement learning of motor skills in high dimensions: a path integral approach. IEEE Int Conf Robot Autom (ICRA). https://doi.org/10.1109/ROBOT.2010.5509336
41. Tuong D, Peters J (2011) Learning task-space tracking control with kernels. IEEE/RSJ Int Conf Intell Robots Syst. https://doi.org/10.1109/IROS.2011.6094428
42. Wiering MA, van Hasselt H (2007) Two novel on-policy reinforcement learning algorithms based on TD(λ)-method. In: Proceedings of the 2007 IEEE symposium on approximate dynamic programming and reinforcement learning (ADPRL). https://doi.org/10.1109/ADPRL.2007.368200

43. Wiering MA, van Hasselt H (2009) The QV family compared to other reinforcement learning algorithms. In: 2009 IEEE symposium on adaptive dynamic programming and reinforcement learning. https://doi.org/10.1109/ADPRL.2009.4927532
44. Xian B, de Queiroz M, Dawson D, Walker I (2004) Task-space tracking control of robots manipulators via quaternion feedback. IEEE Trans Robot Autom 20(1):160–167. https://doi.org/10.1109/TRA.2003.820932
45. Yu W, Perrusquía A (2019) Simplified stable admittance control using end-effector orientations. Int J Soc Robot. https://doi.org/10.1007/s12369-019-00579-y
46. Zhang D, Wei B (2017) On the development of learning control for robotic manipulators. Robotics. https://doi.org/10.3390/robotics6040023
47. Zheng Y, Ma J, Wang L (2017) Consensus of hybrid multi-agent systems. IEEE Trans Neural Netw Learn Syst 29(4):1359–1365. https://doi.org/10.1109/TNNLS.2017.2651402
48. Zhu Y, Li S, Ma J, Zheng Y (2018) Bipartite consensus in networks of agents with antagonistic interactions and quantization. IEEE Trans Circuits Syst II Express Briefs 65(12):2012–2016. https://doi.org/10.1109/TCSII.2018.2811803

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.